

Sistema de Gestión de Tickets (NCP Tickets)

Documentación Técnica de Implementación y Arquitectura del Sistema

Patrón: Clean Architecture (Router Pattern) **Runtime:** Python 3.14.3 **Framework:** FastAPI

ORM: SQLAlchemy 2.0 (Async) **Base de Datos:** Neon DB (PostgreSQL Serverless)

1. Estructura de Directorios del Backend

El backend se encuentra completamente desacoplado y estructurado con separación estricta de responsabilidades:

```
backend/
├── venv/                                # Entorno virtual aislado con Python 3.14.3
├── requirements.txt                     # Dependencias fijadas del proyecto
├── .env.example / .env                 # Plantilla y configuración de variables de entorno
├── alembic.ini & alembic/              # Migraciones versionadas asíncronas con Alembic
├── sql/triggers.sql                     # Triggers PostgreSQL para control automático de fechas en Neon DB
├── app/
│   ├── main.py                         # Instancia FastAPI, CORS, Lifespan y Health Check
│   ├── core/                           # Configuración, Database pool Neon y Seguridad (Google/JWT)
│   ├── models/                         # Modelos ORM SQLAlchemy 2.0 (Usuario, Ticket, Comentario)
│   ├── schemas/                       # DTOs Pydantic v2 (Create, Update, Response)
│   ├── dependencies/                   # Inyección de dependencias (Sesión BD, Auth, Roles)
│   ├── services/                       # Capa de Lógica de Negocio (Autoasignación, Fechas, Auth)
│   └── routers/                       # Controladores HTTP (/auth, /tickets, /comentarios, /usuarios)
```

2. Detalle de Capas y Componentes

2.1. Capa Core (Infraestructura y Conectividad)

- **config.py** : Configuración centralizada vía `pydantic-settings`. Carga variables de entorno (`DATABASE_URL`, `GOOGLE_CLIENT_ID`, `SECRET_KEY`, etc.) con patrón singleton cacheado por `@lru_cache`.
- **database.py** : Crea el motor `create_async_engine` usando el driver `asyncpg`. Incluye configuraciones de alto rendimiento diseñadas para PostgreSQL serverless (Neon DB):
 - `pool_pre_ping=True` : Comprueba la salud del socket antes de ejecutar consultas para evitar cortes si Neon suspende el cómputo.
 - `pool_recycle=300` : Recicla conexiones inactivas tras 5 minutos.
- **security.py** : Validación asíncrona de Google ID Token (Google Workspace SSO) con `httpx`, y emisión/decodificación de tokens JWT internos firmados con algoritmo `HS256`.

2.2. Capa Models (Persistencia ORM SQLAlchemy 2.0)

Implementa la sintaxis moderna `Mapped[]` y `mapped_column()` sin elementos legacy:

- **Usuario (tabla usuarios):** Campos `id`, `nombre`, `email` (único), `rol` (con `CheckConstraint` en `'Usuario'`, `'Soporte'`, `'Administrador'`), `cargo` y `area`.
- **Ticket (tabla tickets):** Campos `id`, `titulo`, `descripcion`, `ubicacion`, `tipo_caso`, `estado` (`'Creado'`, `'En Ejecución'`, `'Solucionado'`), llaves foráneas a `usuarios.id`, fechas con zona horaria (`DateTime(timezone=True)`) y relaciones asíncronas con `lazy="selectin"`.
- **Comentario (tabla comentarios):** Relaciones con borrado en cascada hacia `Ticket` y `Usuario`.
- **Seguridad de Tipos:** Utiliza bloques `if TYPE_CHECKING:` para resolver autocompletado en el IDE sin dependencias circulares en runtime.

2.3. Capa Schemas (Validación Pydantic v2)

- **Separación Estricta:** Schemas de entrada (ej. `TicketCreate`) solo exigen datos de negocio requeridos. Schemas de actualización (`TicketUpdate`) admiten campos opcionales.
- **Salida ORM:** Todos los modelos de respuesta implementan `model_config = ConfigDict(from_attributes=True)`, convirtiendo directamente instancias ORM de SQLAlchemy a JSON.
- **Enums Fuertes:** `RolUsuario` y `EstadoTicket` garantizan que no ingresen valores fuera de regla.

2.4. Capa Dependencies (Inyección y Seguridad)

- `get_db`: Generador asíncrono que abre la sesión, la inyecta al endpoint y realiza `commit()` o `rollback()` automático ante errores.
- `get_current_user`: Extrae el JWT del header `Authorization: Bearer <token>`, valida la firma y consulta al usuario en la BD.
- `require_role(*allowed_roles)`: Fábrica de dependencias de autorización. Retorna inmediatamente HTTP `403 Forbidden` si el usuario no cuenta con los privilegios adecuados.

2.5. Capa Services (Lógica de Negocio)

- **Balanceo de Carga de Soporte:** La función `_auto_assign_soporte()` calcula la cantidad de tickets activos (no solucionados) por cada miembro de soporte y le asigna el nuevo ticket a quien tenga menor carga de trabajo.
- **Control de Solución:** Al actualizar el estado de un ticket a `'Solucionado'`, el servicio fija automáticamente `fecha_solucion = datetime.now(timezone.utc)`.
- **Auto-provisión de Usuarios:** Tras una autenticación válida con Google, si el usuario no existe en la base de datos, se crea con el rol base `'Usuario'`.

3. Catálogo de Endpoints de la API (/api/v1)

Método	Endpoint	Descripción	Acceso / Rol
POST	/api/v1/auth/google	Login con Google ID Token → Emite JWT	Público
GET	/api/v1/auth/me	Perfil del usuario autenticado actual	Autenticado
POST	/api/v1/tickets	Crea ticket y lo autoasigna a soporte	Usuario / Admin
GET	/api/v1/tickets	Lista los tickets creados por el usuario	Autenticado
GET	/api/v1/tickets/dashboard	Dashboard global con filtros de estado/tipo	Soporte / Admin
GET	/api/v1/tickets/{id}	Detalle completo de ticket (valida permisos)	Propietario / Staff
PATCH	/api/v1/tickets/{id}	Actualiza estado o reasigna ticket	Soporte / Admin
POST	/api/v1/tickets/{id}/comentarios	Agrega comentario de seguimiento	Autenticado
GET	/api/v1/tickets/{id}/comentarios	Lista comentarios cronológicos	Autenticado
GET	/api/v1/usuarios	Lista de todos los usuarios registrados	Solo Admin
PATCH	/api/v1/usuarios/{id}	Modifica rol, cargo o área	Solo Admin
GET	/health	Monitoreo de estado de la API	Público

4. Control de Fechas en Neon DB (PostgreSQL)

La integridad de timestamps se garantiza a nivel de motor de base de datos mediante triggers (ubicados en `sql/triggers.sql`):

- **trg_tickets_update_fecha_modificacion** : En cada sentencia `UPDATE` sobre la tabla `tickets`, sobrescribe automáticamente `fecha_modificacion = NOW()`.
- **trg_comentarios_update_ticket_fecha** : En cada sentencia `INSERT` en `comentarios`, ejecuta un `UPDATE` en el ticket padre para refrescar su `fecha_modificacion`.

5. Comandos de Despliegue y Pruebas

```
# 1. Activar entorno virtual
cd c:\NCP_TICKETS\backend
.\venv\Scripts\Activate.ps1

# 2. Ejecutar servidor de desarrollo
uvicorn app.main:app --reload --port 8000

# 3. Documentación interactiva
# Swagger UI: http://localhost:8000/docs
# ReDoc:      http://localhost:8000/redoc
# HealthCheck: http://localhost:8000/health
```

Nota de Configuración en Neon DB

Asegúrate de colocar tu cadena de conexión en `backend/.env` en el parámetro `DATABASE_URL` con el prefijo asíncrono: `postgresql+asyncpg:// ...`